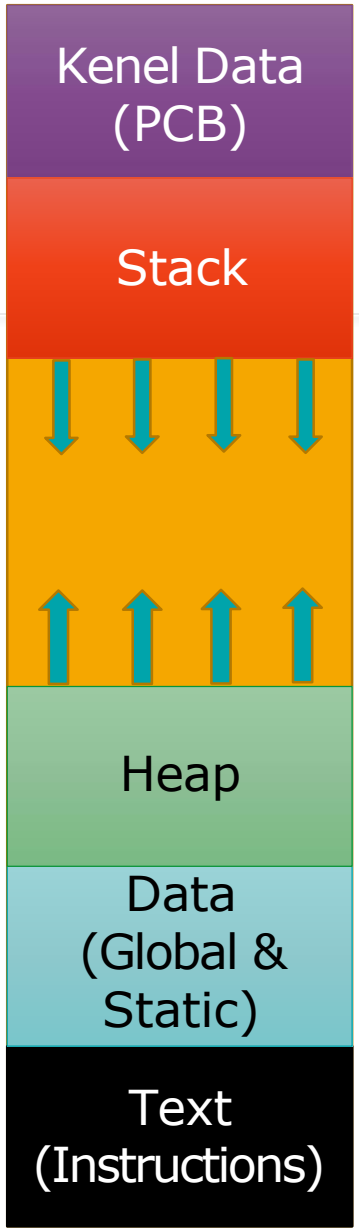
Computer Architecture and Operating Systems
COSC 22073/BECS 22233

Lesson 11: Introduction Memory Management

Department of Statistics & Computer Science
Faculty of Science
University of Kelaniya

Process in an OS

- Process is an execution version of a computer program.
- Process Memory map must present in the main memory.



How to Allocate Main Memory to a Process?

- There are two types of memory allocations techniques:
  1. Contiguous Memory Allocation
  2. Non-Contiguous Memory Allocation

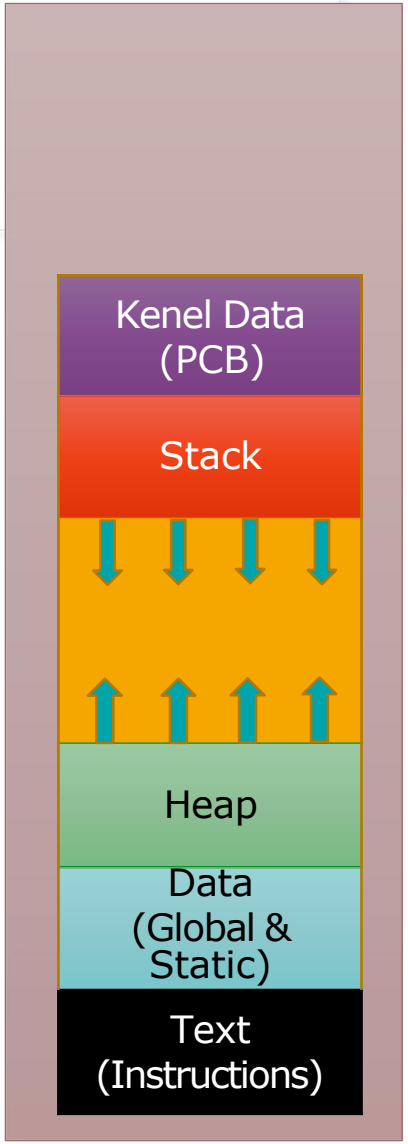
Contiguous Memory Allocation

- When the entire program is stored at a single continuous memory location.

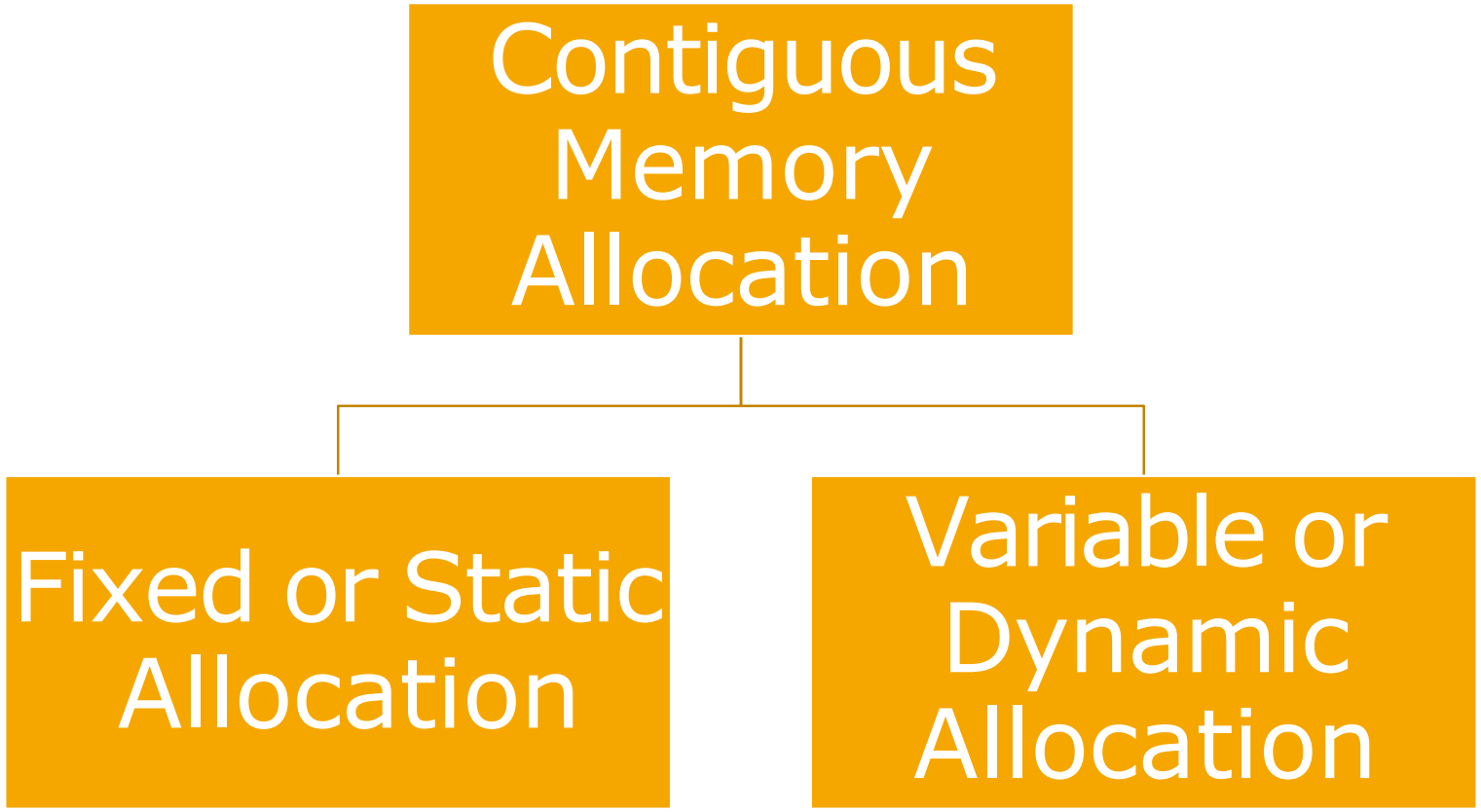
- It is known as centralized approach.

Contiguous Memory Allocation

- Initial Idea:
  - Allocate one process at a time to main memory.
  - Not effiecient.
  - Not supporting for multitasking.

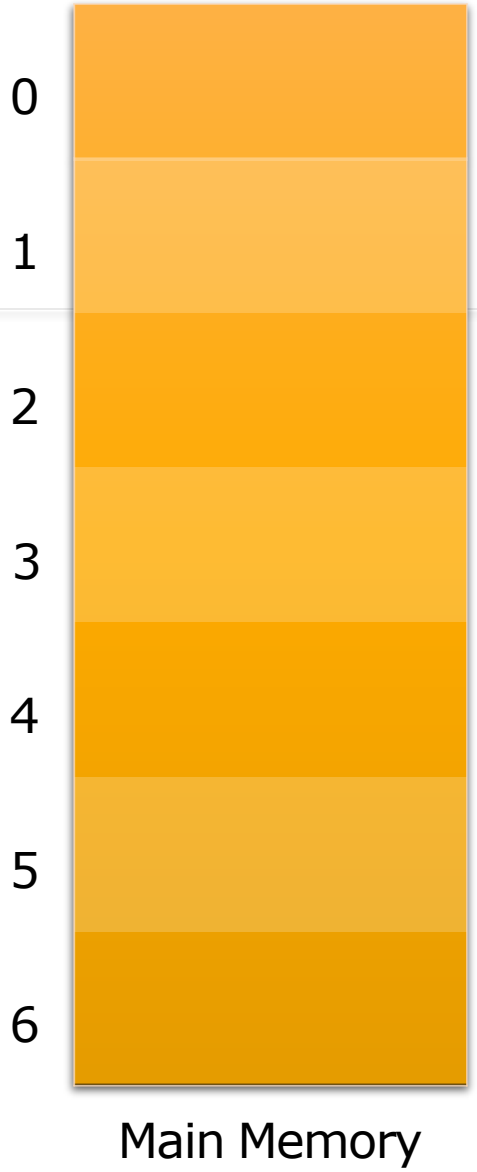


Types of Contiguous Memory Allocation



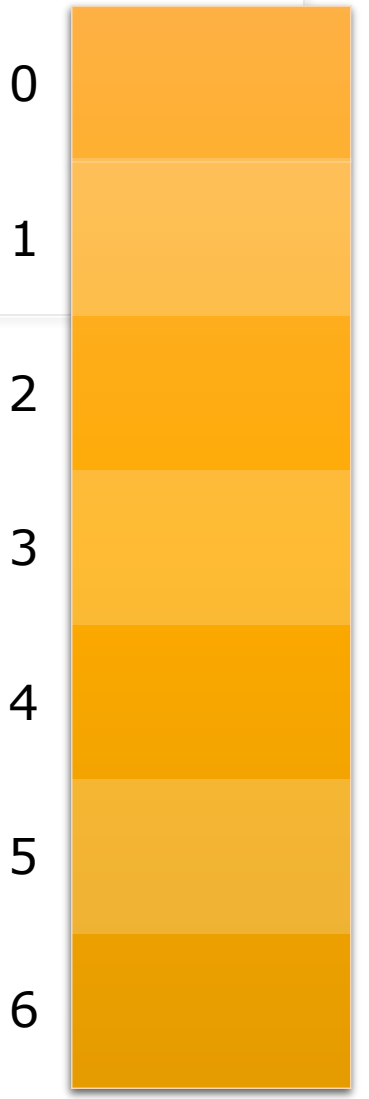
Fixed or Static Partition Contiguous Memory Allocation

- All sectors of memory are of same size (For eg: 1kb)
- The degree of multi-programming depends on the number of partitions.
- When a partition is free, a process is brought from the ready queue and loaded in the memory.
- When a process terminates the partition becomes available for another process.



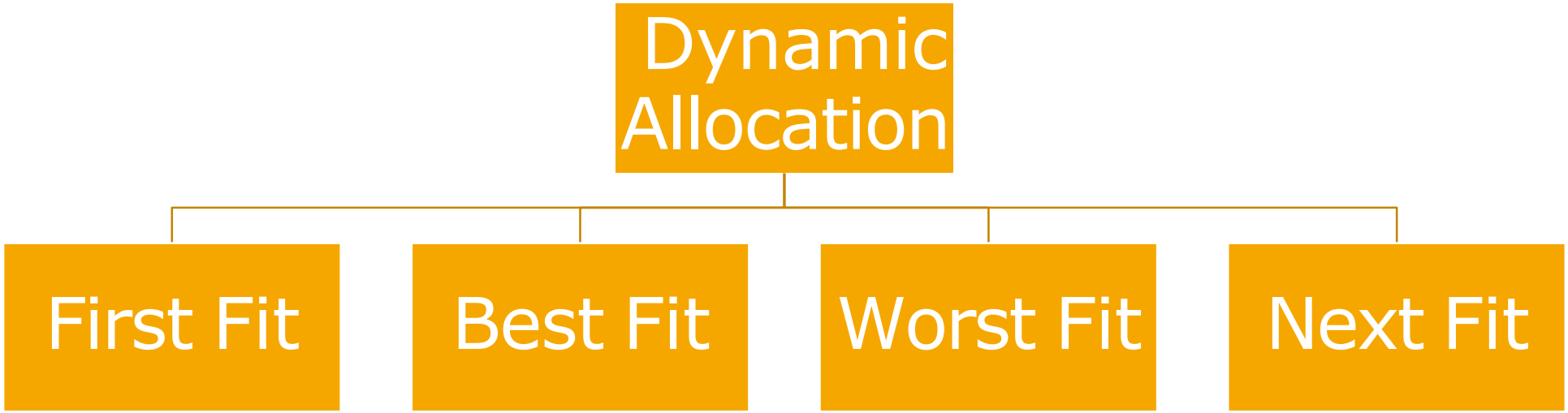
Variable (Dynamic) Partition Contiguous Memory Allocation

- All sectors of memory are of different sizes.
  - Say 1KB, 2 KB, some of 10KB and called as **Hole**.
- OS keeps a table which keeps track of which sector of memory are free.
- Process comes and search the appropriate **hole** and get settled in it.
- There are different dynamic storage allocation techniques.



Main Memory

Dynamic Storage Allocation Techniques



First Fit

- Allocate the first hole that is big enough.
- Searching starts at the begining of the memory.
- Stop the search as soon as it finds a free hole that is large enough.

Example

- On a system using first fit allocation, assume memory is allocated as specified in the figure before the additional requests for 20 KB, 10 KB, and 5KB (in that order) are received. Specify which location will be allocated by the OS for each of the new requests.

Used 10 KB	Hole 10 KB	Used 20 KB	Hole 30 KB	Used 10 KB	Hole 5 KB	Used 30 KB	Hole 20 KB	Used 10 KB	Hole 15 KB	Used 20 KB	Hole 20 KB
A	B	C	D	E	F	G	H	I	J	K	L

Wrost Fit

- Allocate the largest hole.
- Search the entire list.
- This strategy produces the largest leftover hole which may be more useful than the smaller leftover hole from a best-fit approach.

Example

- On a system using wrost fit allocation, assume memory is allocated as specified in the figure before the additional request for 20 KB, 10 KB, and 5KB (in that order) are received. Specify the which location will be allocated by the OS for each of the new requests.

Used 10 KB	Hole 10 KB	Used 20 KB	Hole 30 KB	Used 10 KB	Hole 5 KB	Used 30 KB	Hole 20 KB	Used 10 KB	Hole 15 KB	Used 20 KB	Hole 20 KB
A	B	C	D	E	F	G	H	I	J	K	L

Best Fit

- Allocate the smallest hole that is big enough.
- We must search the entire list, unless the list is ordered by size.
- This strategy produces the smallest leftover hole.

Example

- On a system using best fit allocation, assume memory is allocated as specified in the figure before the additional request for 20 KB, 10 KB, and 5KB (in that order) are received. Specify the which location will be allocated by the OS for each of the new requests.

Used 10 KB	Hole 10 KB	Used 20 KB	Hole 30 KB	Used 10 KB	Hole 5 KB	Used 30 KB	Hole 20 KB	Used 10 KB	Hole 15 KB	Used 20 KB	Hole 20 KB
A	B	C	D	E	F	G	H	I	J	K	L

Next Fit

- In the first fit, since all the search starts from the beginning of memory, holes at the beginning of memory are considered for allocation more frequently.
- In the next fit, it keeps track of which hole was allocated.
- The next search starts at the last hole allocated, not at the beginning of the memory.
- The next fit is a modified version of the ‘first fit’. It begins as the first fit to find a free partition but when called next time it starts searching from where it left off, not from the beginning

Example

- On a system using next fit allocation, assume memory is allocated as specified in the figure before the additional request for 20 KB, 10 KB, and 5KB (in that order) are received. Specify the which location will be allocated by the OS for each of the new requests.

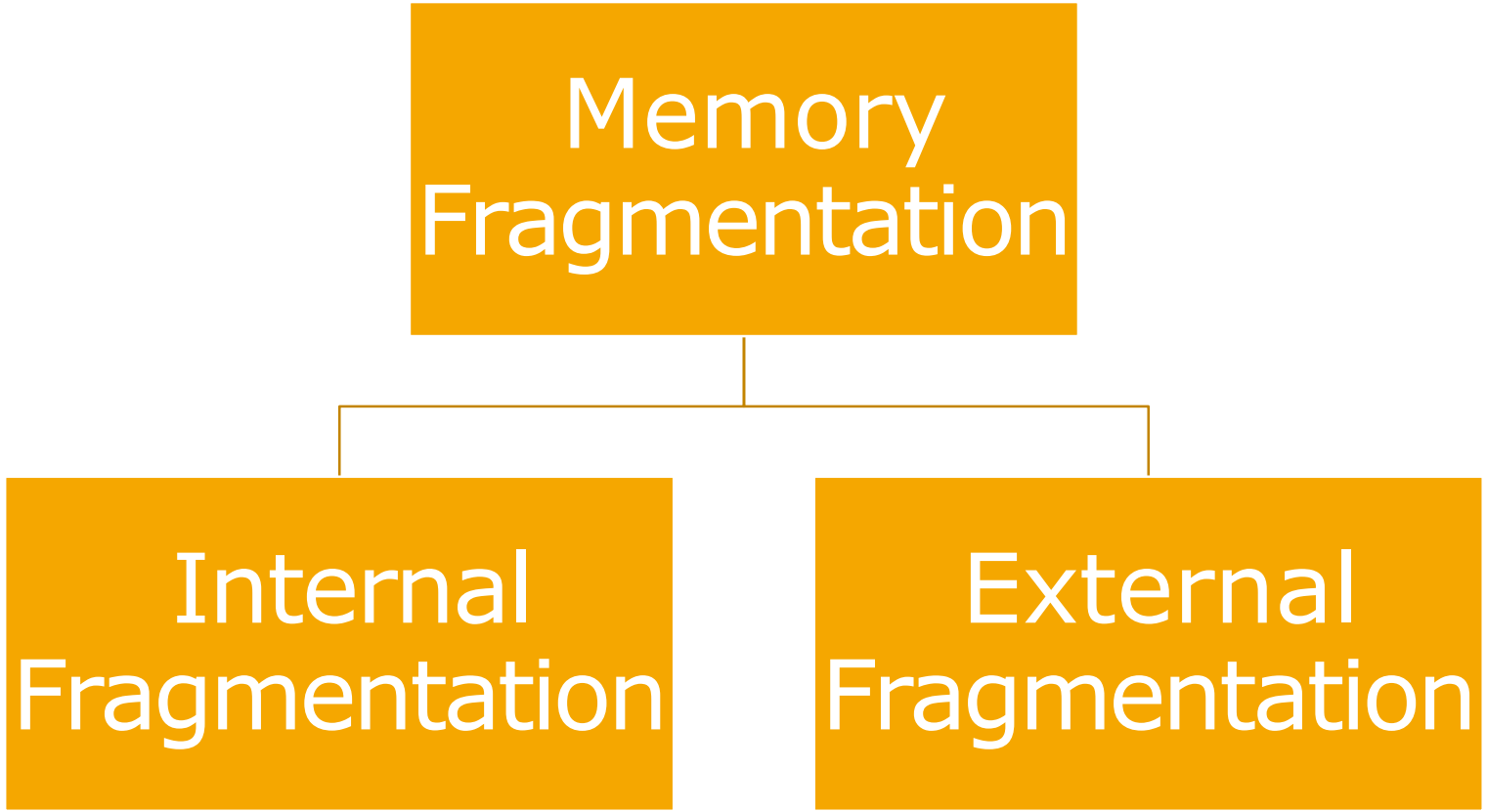
Used 10 KB	Hole 10 KB	Used 20 KB	Hole 30 KB	Used 10 KB	Hole 5 KB	Used 30 KB	Hole 20 KB	Used 10 KB	Hole 15 KB	Used 20 KB	Hole 20 KB
A	B	C	D	E	F	G	H	I	J	K	L

Memory Fragmentation

- Memory Fragmentation is when most of your memory is allocated in a large number of non-contiguous blocks: leaving a good percentage of your total memory unallocated, but unusable.

- That is, fragmentation is the inability to use memory even if it is free.

Main Types of Fragmentation



Internal Fragmentation

- Due to the rules governing memory allocation, more computer memory is sometimes allocated than needed.

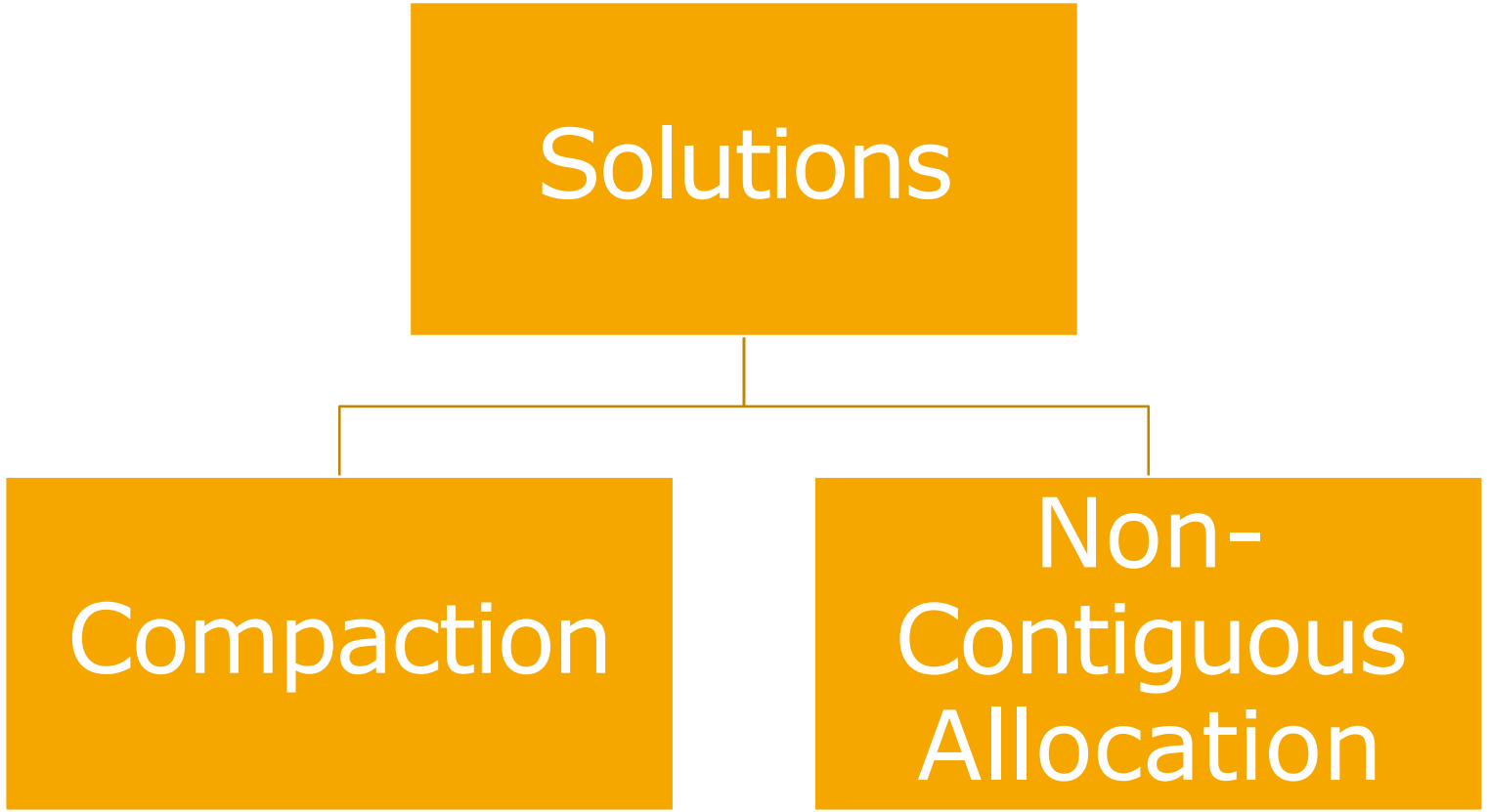
- Therefore, some portions of memory is left unused as it cannot be used by another process.

External Fragmentation

- This arises when free memory is separated into small blocks and is inter separated by allocated memory.

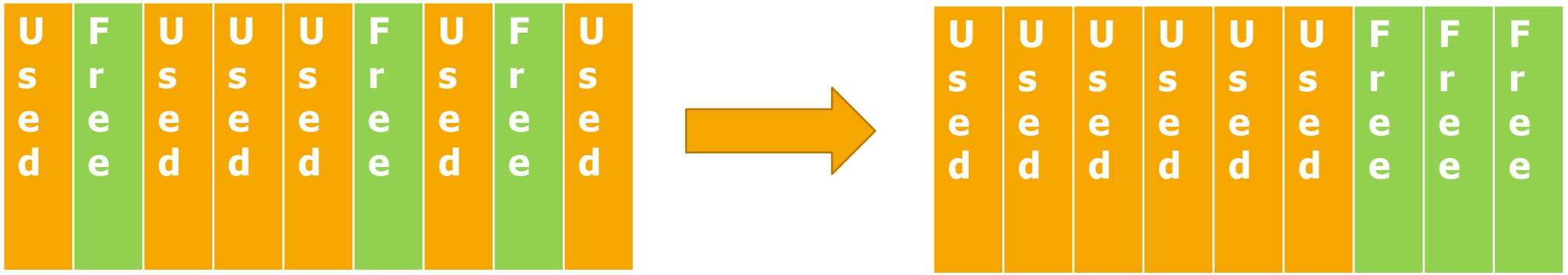
- That is, total memory is enough to satisfy a request or to reside a process in it, but it is not contiguous.

Solutions for Fragmentation



Compaction

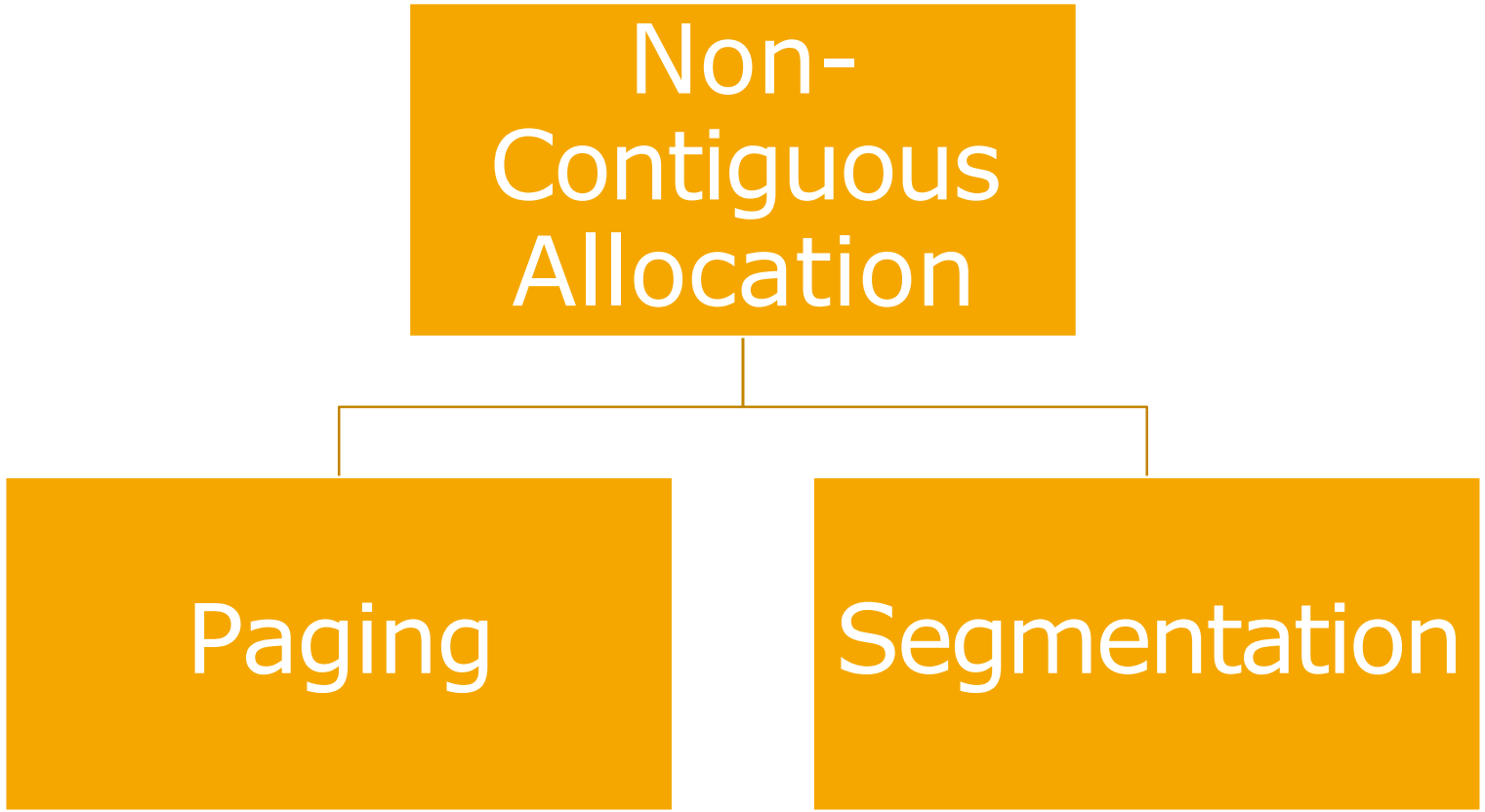
- Compaction is a process in which free spaces are collected to a large memory chunks. That is, combine all the empty spaces together.



Compaction

- Processes are on one side and free space is on the other side in the memory.
- During memory compaction:
  - Block processes for compaction
  - Move them towards lower address space
  - Allocate memory for pending processes
  - Resumed blocked processes
- It comsumes CPU time.

Non-Contiguous Allocation



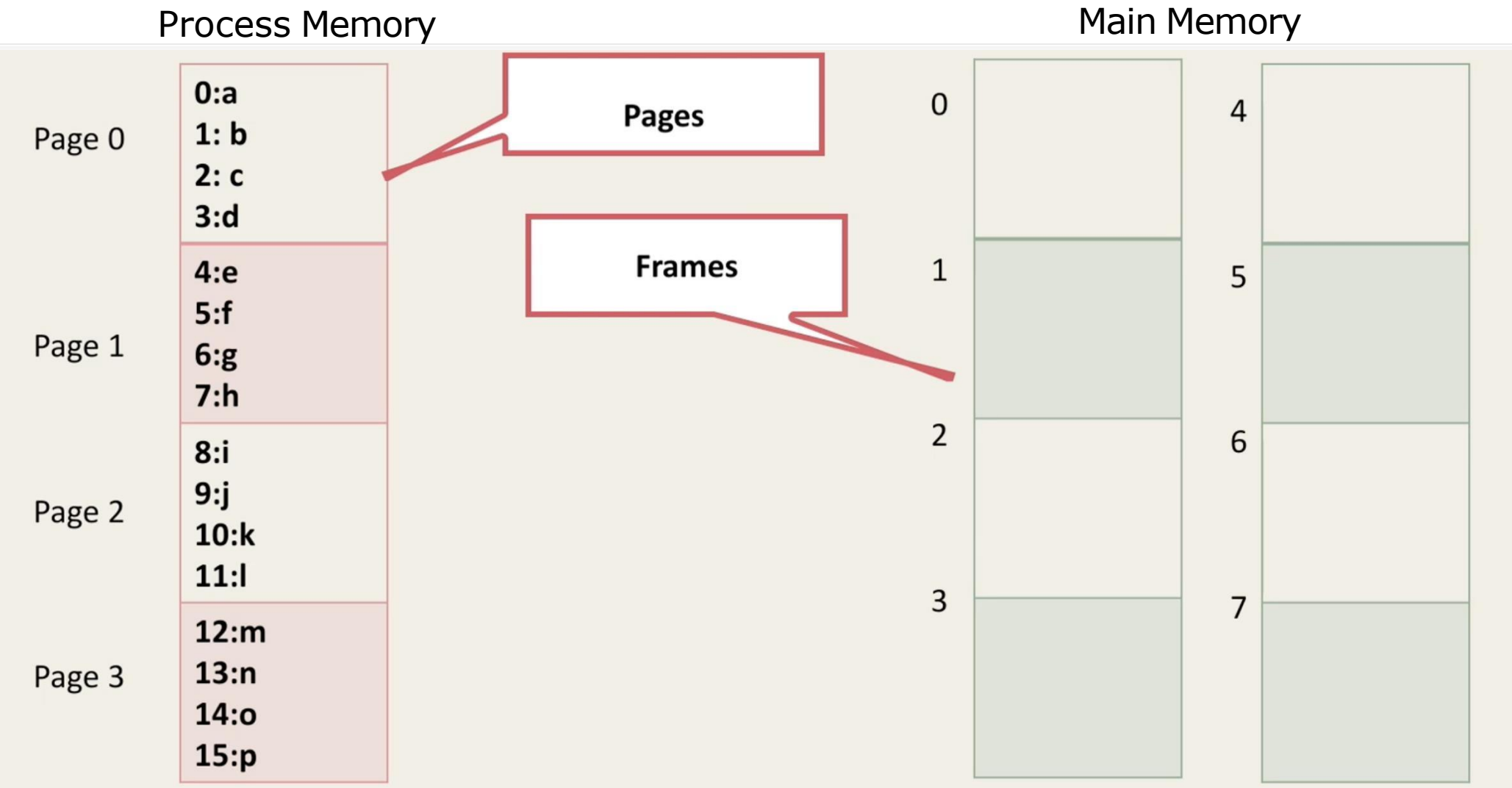
Paging

- Memory Management Technique that permits the physical address space of a process to be non-contiguous.

- Physical memory is divided into fixed size-block called **FRAMES**.

- Logical memory is divided into blocks of the same size called **PAGES**.

Paging Model



Paging: Address Translation

Process Memory

Page 0	0000	0:a
	0001	1: b
	0010	2: c
	0011	3:d
Page 1	0100	4:e
	0101	5:f
	0110	6:g
	0111	7:h
Page 2	1000	8:i
	1001	9:j
	1010	10:k
	1011	11:l
Page 3	1100	12:m
	1101	13:n
	1110	14:o
	1111	15:p

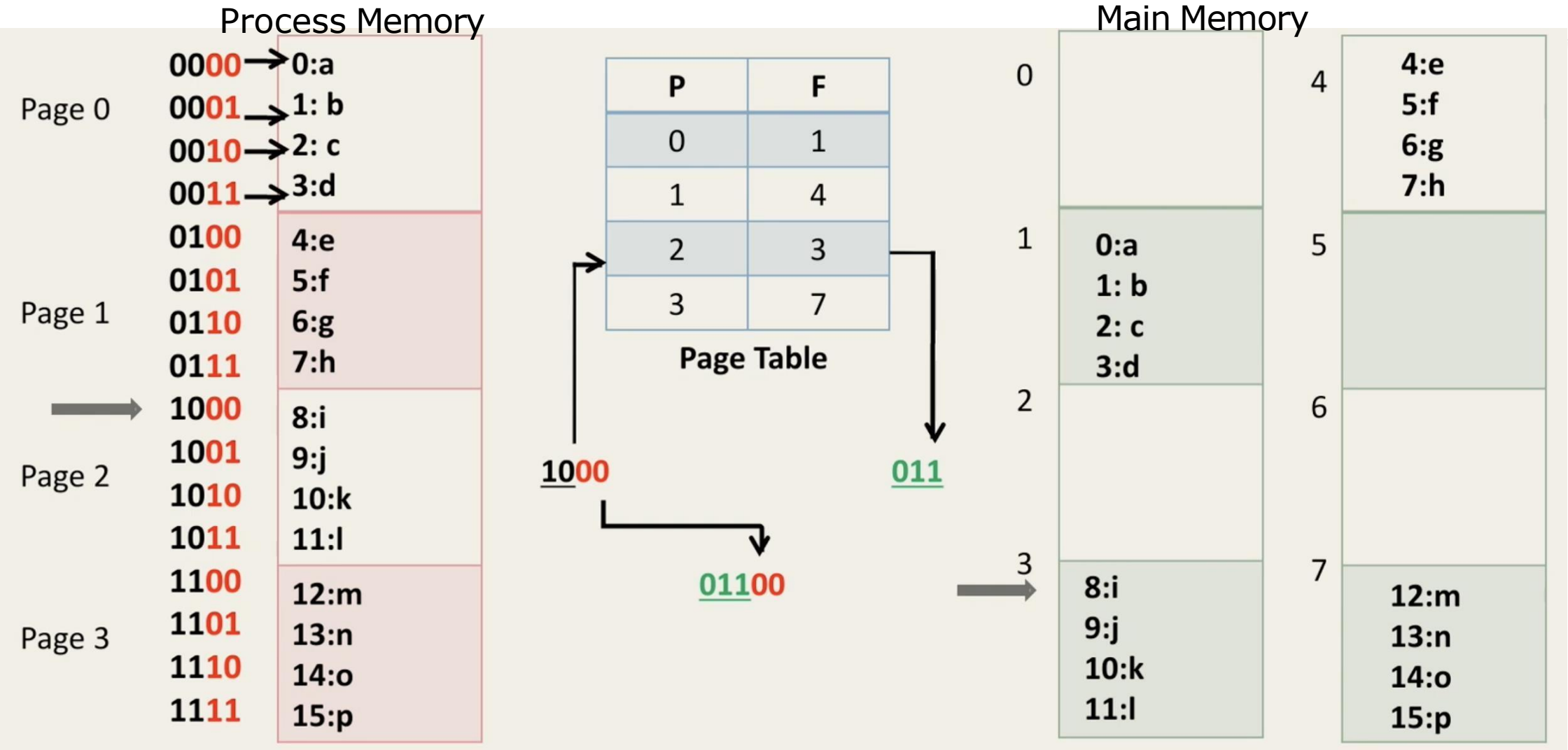
P	F
0	1
1	4
2	3
3	7

Page Table

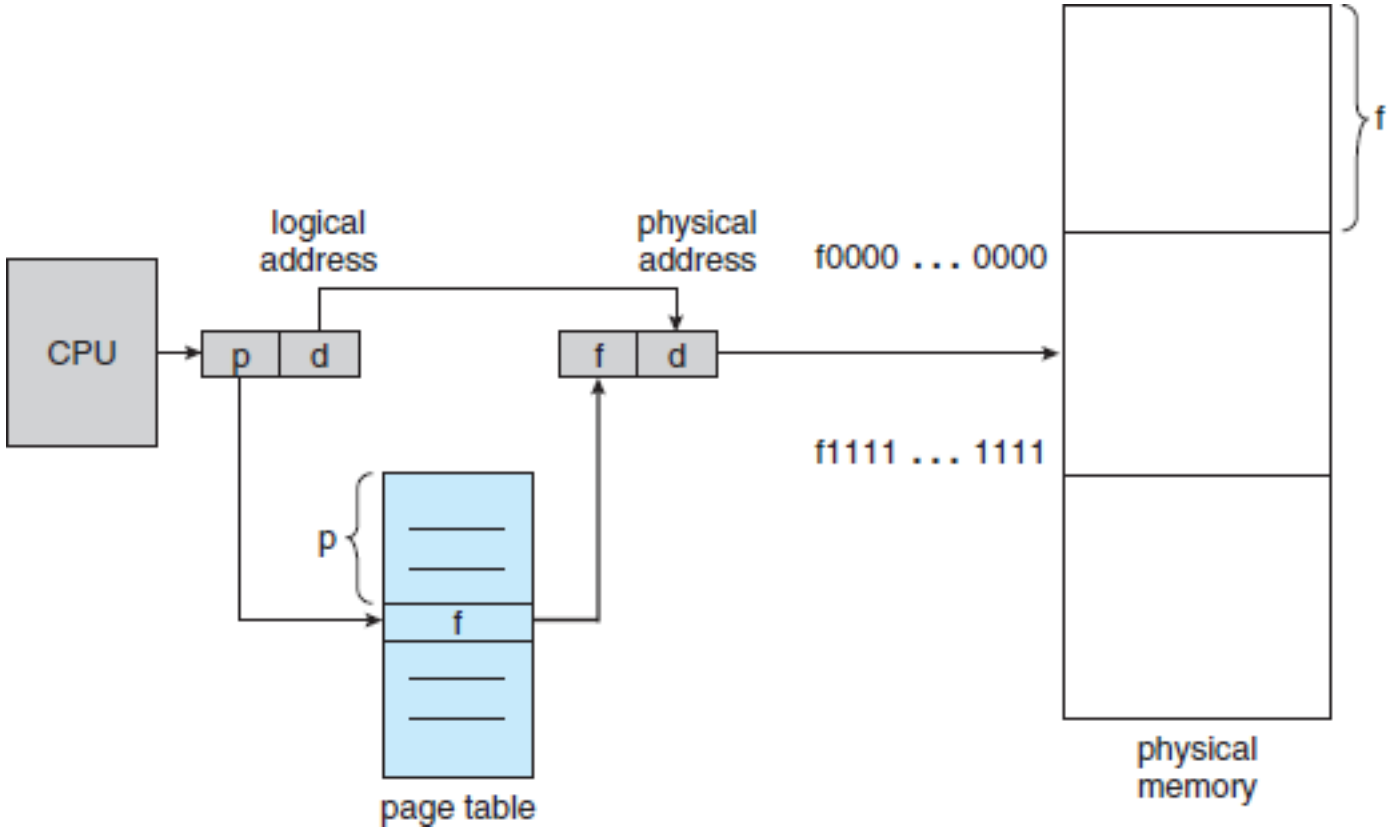
Main Memory

0	
1	0:a 1: b 2: c 3:d
2	
3	8:i 9:j 10:k 11:l
4	4:e 5:f 6:g 7:h
5	
6	
7	12:m 13:n 14:o 15:p

Paging: Address Translation



Paging: Hardware



Logical Address = Page Number + Offset

Physical Address = Frame Number + Offset

Page Table

- Each process has its own page table stored in memory.
- Page table contains frame numbers
- To make usage more efficient the page table is kept in the main memory & Page Table Base Register (PBTR)

Page Table

- Page table implementation maintains a four-bit per page table entry
  - Use bit: It is a flag associated with a page in memory that indicates whether the page has been accessed (read from or written to) during a certain period.
  - Modified bit: is a flag associated with a memory page that indicates whether the contents of the page have been modified (written to) since it was last loaded into memory or since the bit was last cleared.
  - Valid bit:is a flag associated with a page table entry that indicates whether the corresponding page in memory is currently valid or not.
  - Read-only bit:is a flag associated with a page table entry or a page in memory that indicates whether the contents of the page are read-only or writable.

Example:

- Consider a system in which logical address equal to 27 bits and the physical address requires 21 bits. The page size is 4K words. Calculate the number of pages and number of frames.

Answer

1. **Calculate the Number of Pages:**
   - Given that the logical address space is 27 bits and the page size is 4K words, you can find the number of pages using the formula:

     $$\text{Number of Pages} = \frac{\text{Logical Address Space}}{\text{Page Size}}$$

     $$\text{Number of Pages} = \frac{2^{27}}{2^{12}}$$

     $$\text{Number of Pages} = 2^{27-12} = 2^{15}$$

     Therefore, the number of pages is 2^{15} or 32,768 pages.
2. **Calculate the Number of Frames:**
   - Given that the physical address requires 21 bits, the number of frames is determined by the formula:

     $$\text{Number of Frames} = \frac{\text{Physical Address Space}}{\text{Page Size}}$$

     $$\text{Number of Frames} = \frac{2^{21}}{2^{12}}$$

     $$\text{Number of Frames} = 2^{21-12} = 2^{9}$$

     Therefore, the number of frames is 2^{9} or 512 frames.

Paging: Advantages

- No external fragmentation.
- Simple memory management scheme.
- Swapping easy

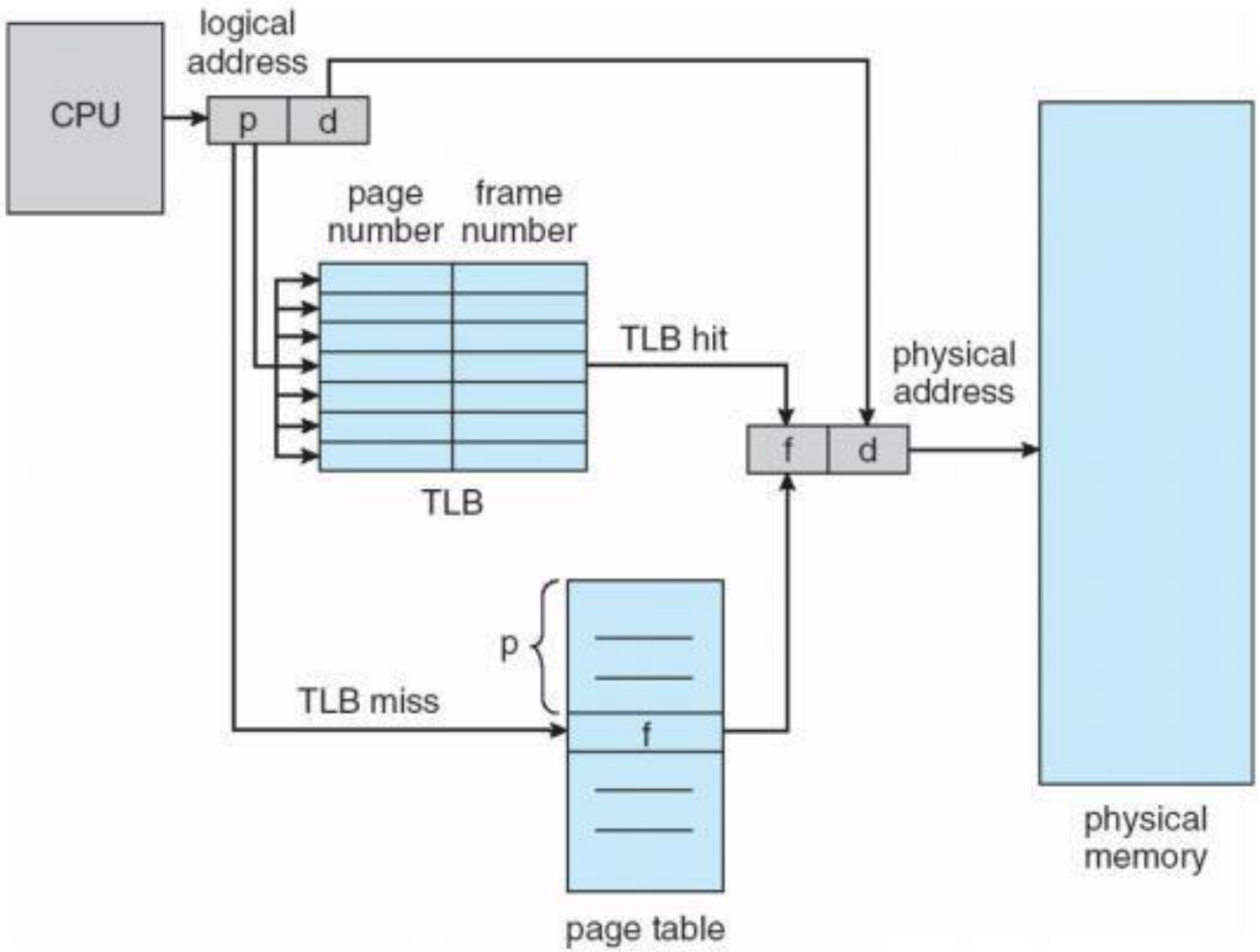
Paging: Disadvantages

- Internal fragmentation
- Page table may consume more memory

Solution to Access Time

- Translation Look-aside Buffer (TLB)
  - It is a small fast cache memory.
  - Reduces effective access time.
  - It consists of two columns:
    - Page Number
    - Frame Number

Paging with TLB



More about TLB

- Only one TLB in the system.
- So, whenever context switch occurs, the entire content of the TLB is flushed and deleted.
- TLB again updated with the currently running process.
- A TLB contains only few page table entries.

More about TLB

- If the page number is found, then it is TLB hit. Otherwise TLB miss.
- The percentage of times that a particular page number is found in the table is called HIT RATIO.

More about TLB

- Effective Memory Access Time =

{Hit Ratio $*$ (access time for the TLB
+access time)
+Miss ration *Total Access time}

Example:

- If it takes 20 nanoseconds to search the TLB and 100 nanoseconds to access the memory. If we fail to find the page in TLB then the access time is 220 nanoseconds. What is the effective memory access time with a hit ration of 80%?

Answer

$$\text{EMAT} = \text{TLB hit ratio} \times (\text{TLB access time} + \text{Memory access time on TLB hit}) + (1 - \text{TLB hit ratio}) \times (\text{TLB access time} + \text{Memory access time on TLB miss})$$

Given the information provided:

- TLB access time = 20 nanoseconds
- Memory access time on TLB hit = 100 nanoseconds
- Memory access time on TLB miss = 220 nanoseconds
- TLB hit ratio = 80% (or 0.8)

Let's substitute these values into the formula:

$$\text{EMAT} = 0.8 \times (20 + 100) + 0.2 \times (20 + 220)$$

$$\text{EMAT} = 0.8 \times 120 + 0.2 \times 240$$

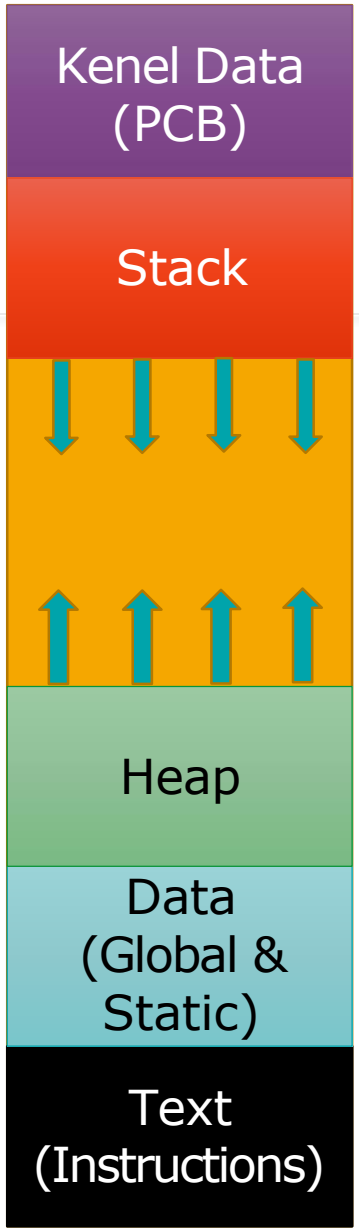
$$\text{EMAT} = 96 + 48$$

EMAT=144 nanoseconds

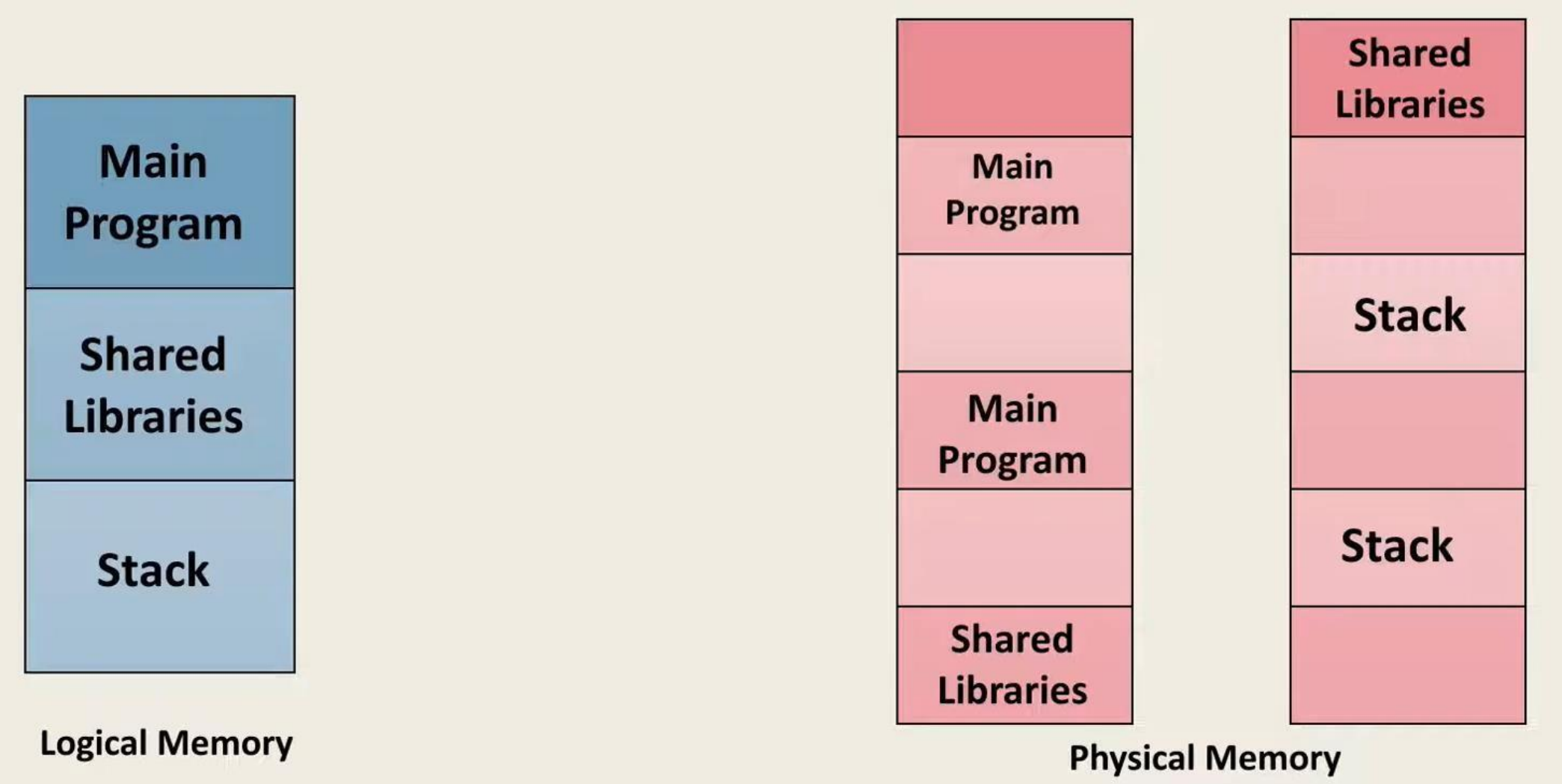
Therefore, the effective memory access time is 144 nanoseconds with a TLB hit ratio of 80%.

Beyond Paging

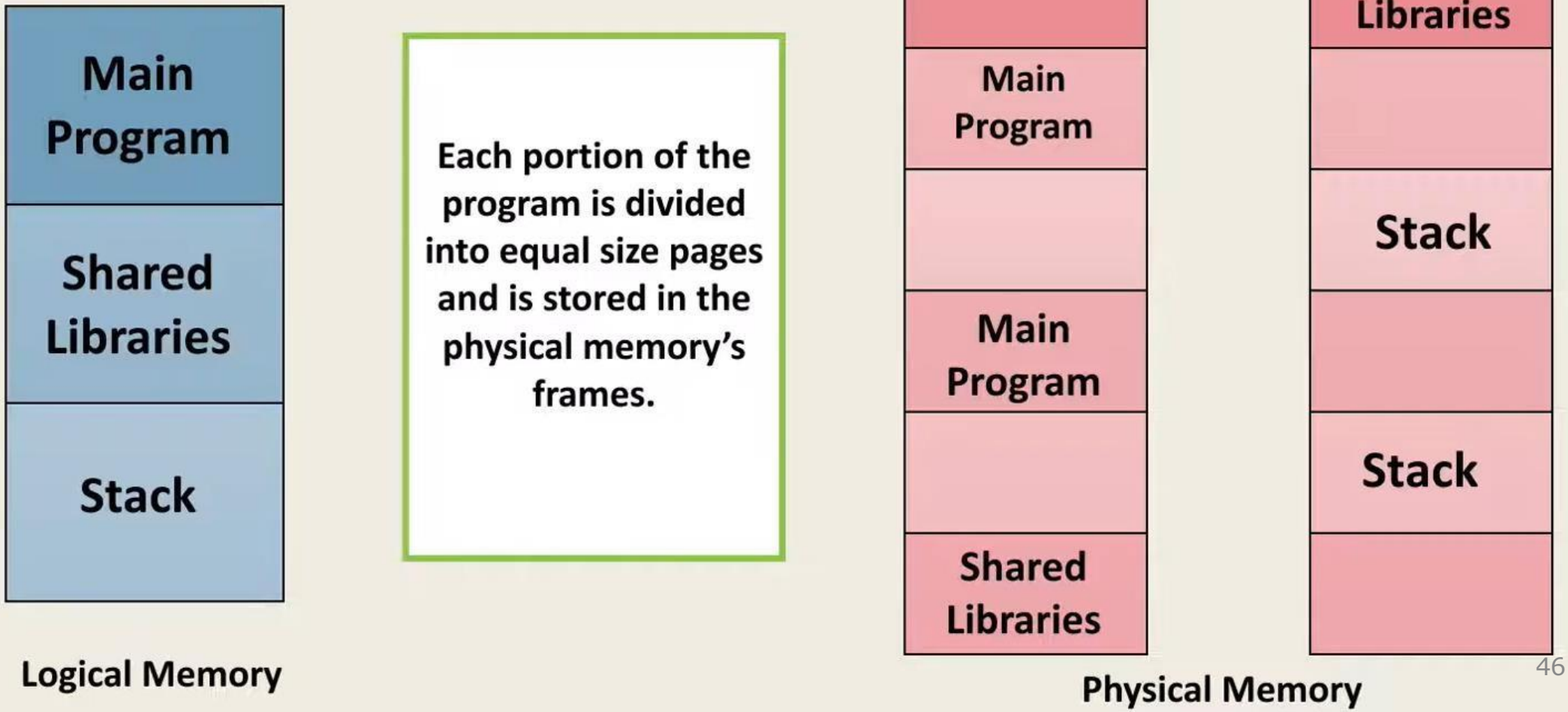
- Paging does not preserve users view of the memory.



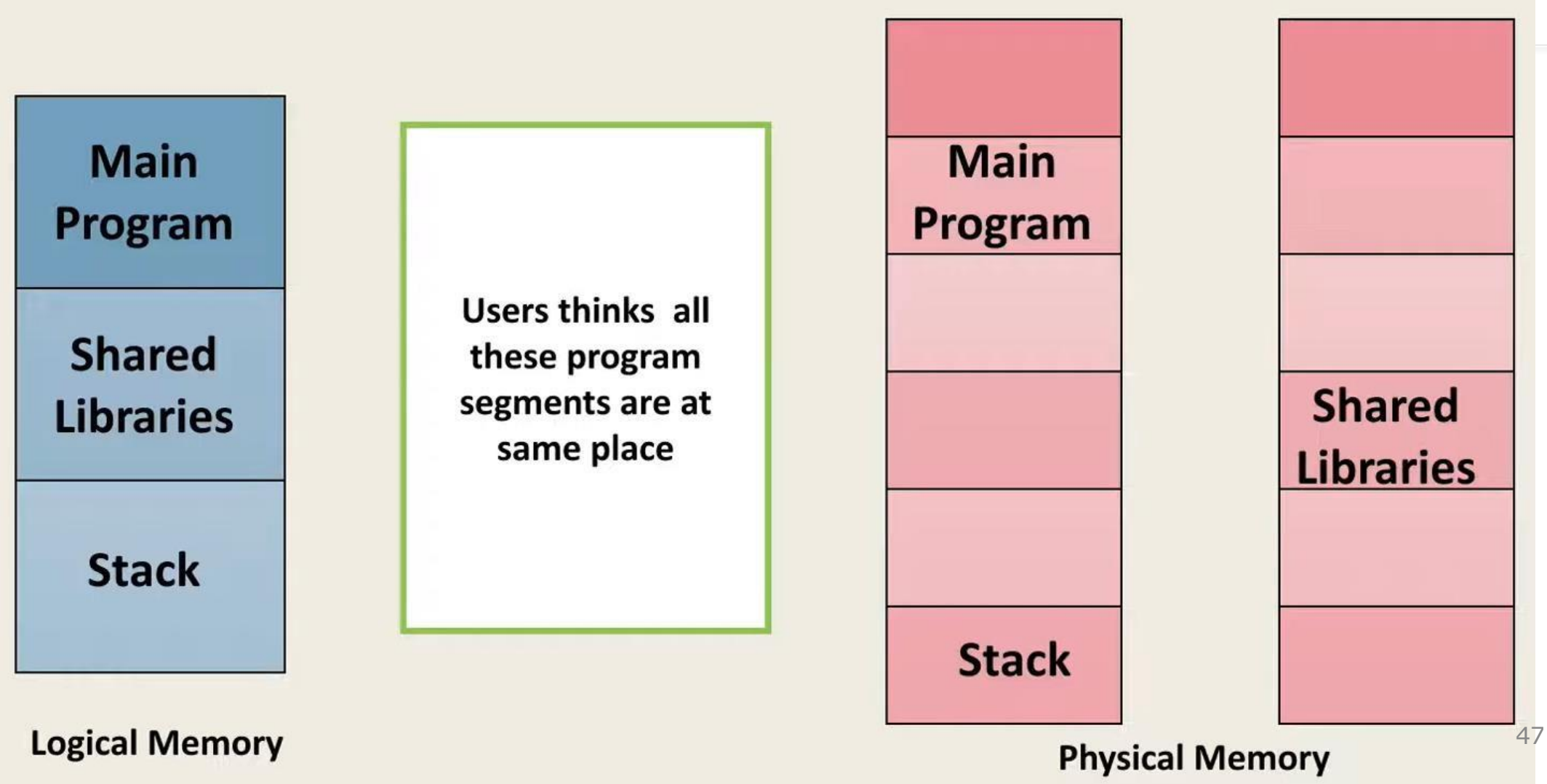
What happened in Paging?



What happened in Paging?



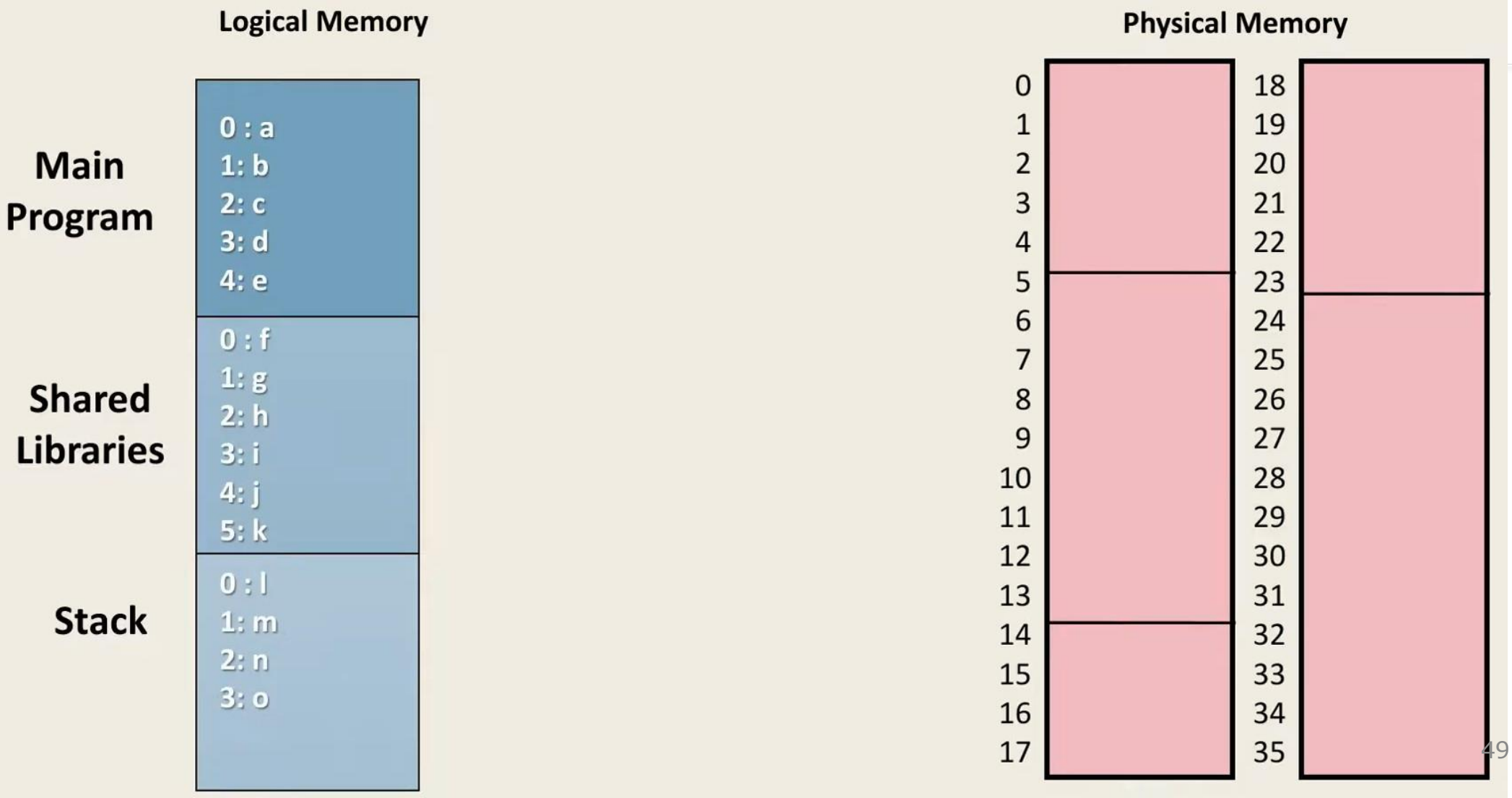
User's View


Main Program
Shared Libraries
Stack
Logical Memory
Users thinks all these program segments are at same place
Main Program
Stack
Shared Libraries
Physical Memory

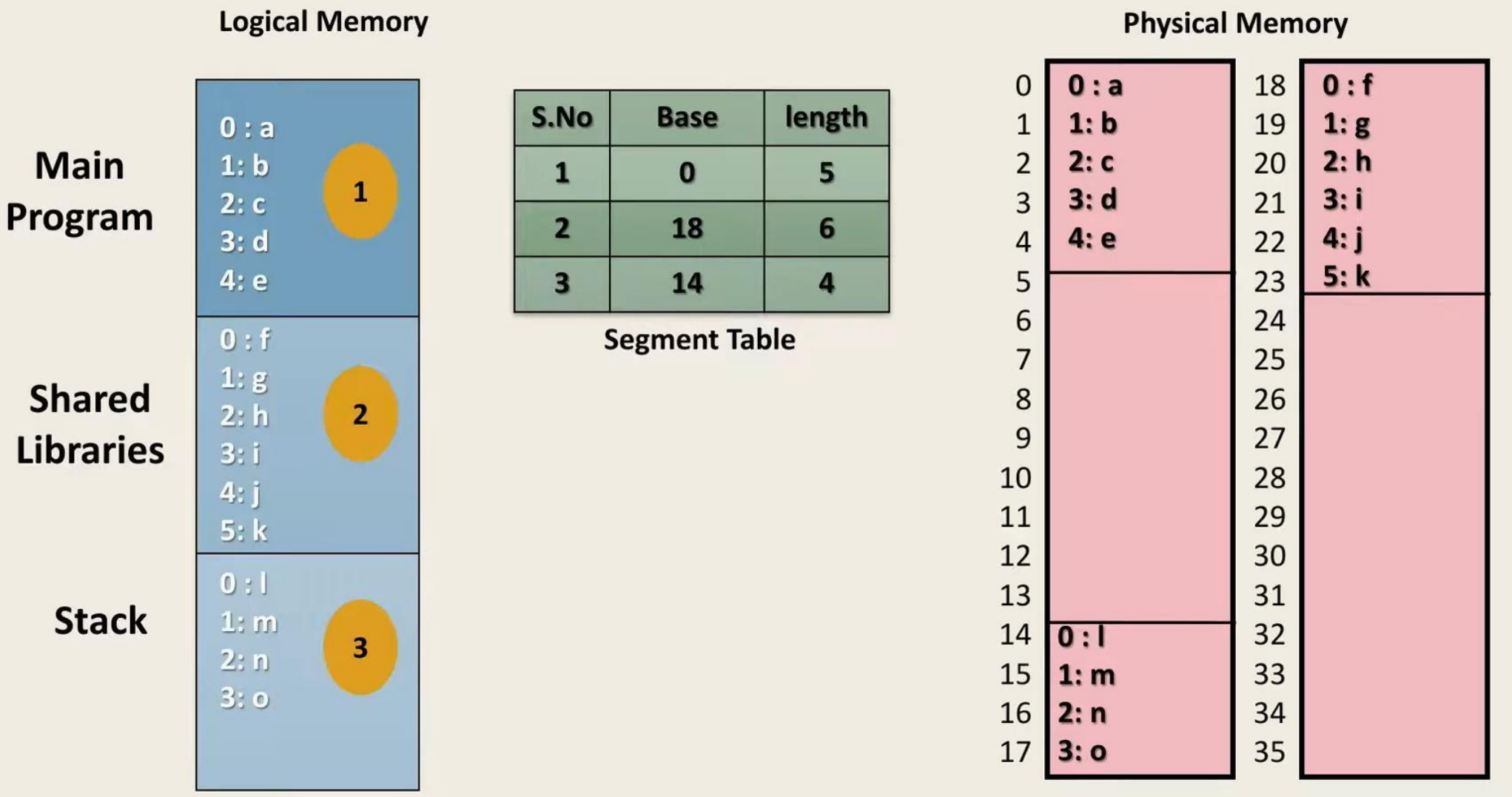
Segmentation

- Segmentation is a memory management technique in which memory is divided into variable sized chunks which can be allocated to processes.
- Each chunk is called a segment.
- A table store the information about all such segments called the segment table.
- Segmentation supports user view of the memory.

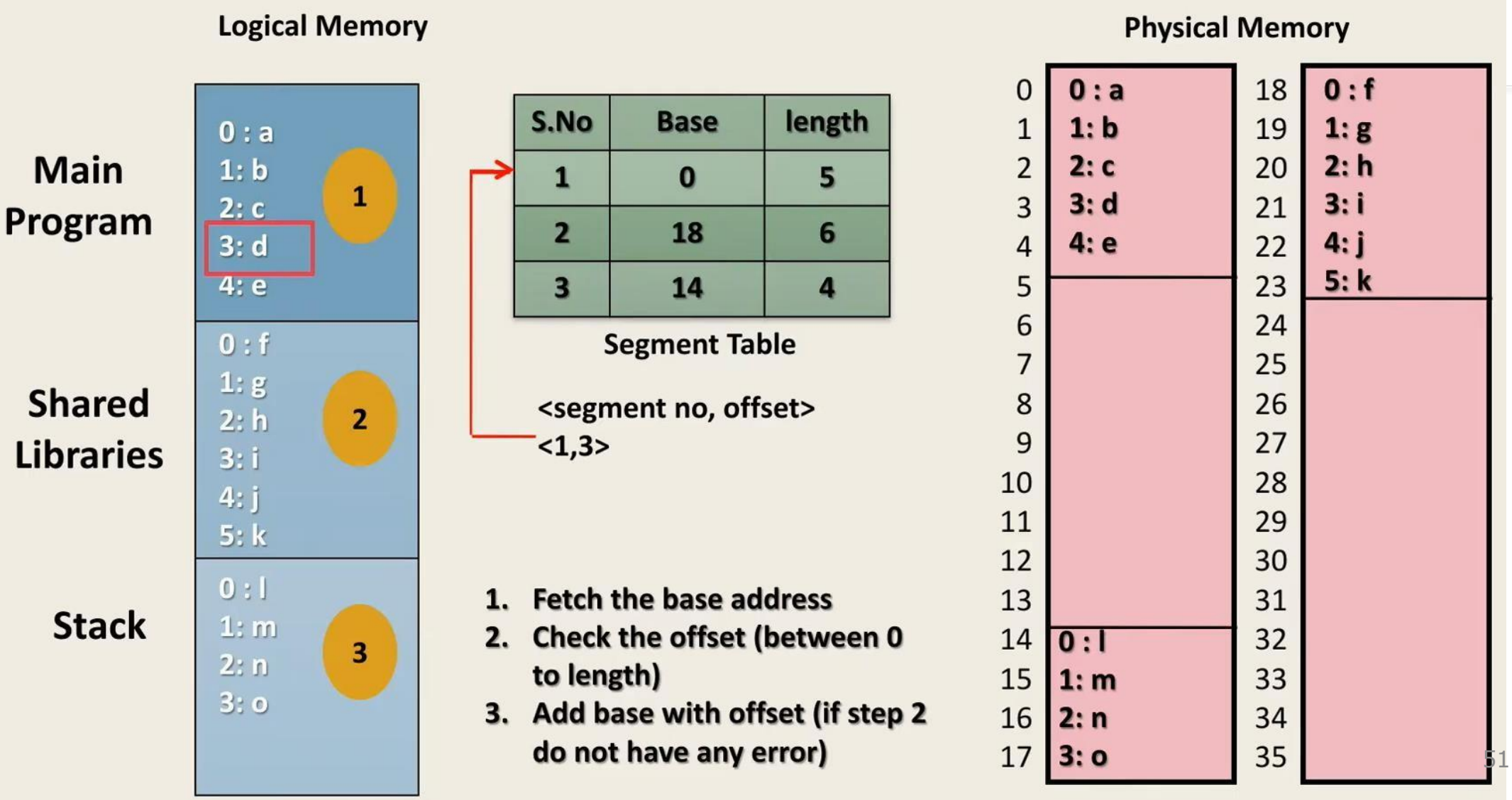
Segmentation



Segmentation



Segmentation


Logical Memory
Main Program
0 : a
1: b
2: c
3: d
4: e
1
Shared Libraries
0 : f
1: g
2: h
3: i
4: j
5: k
2
Stack
0 : l
1: m
2: n
3: o
3
S.No | Base | length
1 | 0 | 5
2 | 18 | 6
3 | 14 | 4
Segment Table
<segment no, offset>
<1,3>
1. Fetch the base address
2. Check the offset (between 0 to length)
3. Add base with offset (if step 2 do not have any error)
Physical Memory
0 : a
1: b
2: c
3: d
4: e
0 : l
1: m
2: n
3: o
0 : f
1: g
2: h
3: i
4: j
5: k

Paged Segmentation

- It utilize both paging and segmentation
- User space is broken into a number of segments
- Each segment is broken into fixed size pages.